

Sistema de gestión de biblioteca

- Mostrar las bases de datos existentes

```
mi_db> show dbs;  
admin      40.00 KiB  
config    100.00 KiB  
local      40.00 KiB  
mi_db      40.00 KiB
```

- Crear y usar una nueva base de datos llamada biblioteca.

```
mi_db> use biblioteca  
switched to db biblioteca  
biblioteca> 
```

- Esto creará la base de datos si aún no existe

- Crear una colección llamada libros

```
biblioteca> db.createCollection("libros")  
{ ok: 1 }  
biblioteca> 
```

- Insertar el siguiente libro con confirmación de escritura:

titulo: Cien Años de Soledad

autor: Gabriel García Márquez

año: 1967

genero: Novela

disponible: true

```
test> use biblioteca
switched to db biblioteca
biblioteca> db.libros.insertOne({titulo: "Cien Años de Soledad", autor:"Gabriel García Márquez", año: 1967, genero: "Novela",disponible:true})
{
  acknowledged: true,
  insertedId: ObjectId('6a31cc7caf9ea9fde59df8a3')
}
biblioteca> □
```

- Insertar múltiples libros con parámetro de insertar en orden y confirmación de escritura

```
biblioteca> db.libros.insertMany([(titulo: "Don Quijote de la Mancha",autor:"Miguel de Cervantes",año: 1605,genero: "Novela",disponible: true),(titulo: "1984",autor: "George Orwell",año: 1949,genero: "Ciencia Ficción",disponible: false),(titulo: "El Principito",autor: "Antoine de Saint-Exupéry",año: 1943,genero: "Novela Corta",disponible: true)]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a31cef3af9ea9fde59df8a4'),
    '1': ObjectId('6a31cef3af9ea9fde59df8a5'),
    '2': ObjectId('6a31cef3af9ea9fde59df8a6')
  }
}
biblioteca> □
```

- Listar todos los libros.

```
biblioteca> db.libros.find();
[
  {
    _id: ObjectId('6a31cc7caf9ea9fde59df8a3'),
    titulo: 'Cien Años de Soledad',
    autor: 'Gabriel García Márquez',
    'año': 1967,
    genero: 'Novela',
    disponible: true
  },
  {
    _id: ObjectId('6a31cef3af9ea9fde59df8a4'),
    titulo: 'Don Quijote de la Mancha',
    autor: 'Miguel de Cervantes',
    'año': 1605,
    genero: 'Novela',
    disponible: true
  },
  {
    _id: ObjectId('6a31cef3af9ea9fde59df8a5'),
    titulo: '1984',
    autor: 'George Orwell',
    'año': 1949,
    genero: 'Ciencia Ficción',
    disponible: false
  },
  {
    _id: ObjectId('6a31cef3af9ea9fde59df8a6'),
    titulo: 'El Principito',
    autor: 'Antoine de Saint-Exupéry',
    'año': 1943,
    genero: 'Novela Corta',
    disponible: true
  }
]
biblioteca> 
```

- Eliminar la base de datos.

```
biblioteca> db.dropDatabase()  
{ ok: 1, dropped: 'biblioteca' }  
biblioteca> 
```

- ¿Qué diferencias hay entre insertOne() e insertMany()?
 - insertOne(): Permite insertar un solo documento.
 - insertMany(): Permite insertar varios documentos al mismo tiempo.
- ¿Qué sucede se usa dropDatabase()?
 - Elimina la base de datos en la cual se haya ejecutado el comando.
- Practicar y corregir los siguientes comandos de inserción de documentos en MongoDB, identificando errores comunes y aplicando la sintaxis correcta.

Analice los siguientes comandos de inserción:

- Identificar los errores en cada línea de código
- Explicar brevemente por qué el comando original no funcionaría
- Corregir los comandos para que funcionen correctamente en MongoDB

Comandos a Corregir:

- Insert.Insert(Nombre:Jose)
R/
 - No existe ese método, además de no tener comillas para identificar el valor.
 - Por que no lleva la estructura correcta que debe ser:
db.collection.insertOne(), además de no llevar comillas
 - db.personas.insertOne({Nombre: "Jose"})
- db.insert({Nombre:"Carlos"})
R/
 - Está incompleto el método
 - Le falta completar bien el insert por qué hay de dos tipos, en bloque y individual, además de que falta a qué collection se está dirigiendo
 - db.personas.insertOne({Nombre:"Carlos"})

- `insert.insert({Nombre:"Jennifer"},{Nombre:"Jose"})`
R/
 - No existe ese método, está incompleto para insertar varios documentos.
 - Para insertar varios documentos se debe usar `insertMany()`
 - `db.personas.insertMany([ {Nombre: "Jennifer"}, {Nombre: "Jose"} ])`
- Realizar un programa en su lenguaje de programación donde realice una inserción y capture el mensaje que la base de datos le retorna, baje el servidor (detenga el servicio) e inserte un nuevo documento. Capture el error desde su programa y capture el mensaje y peguelo en el documento que esta construyendo.

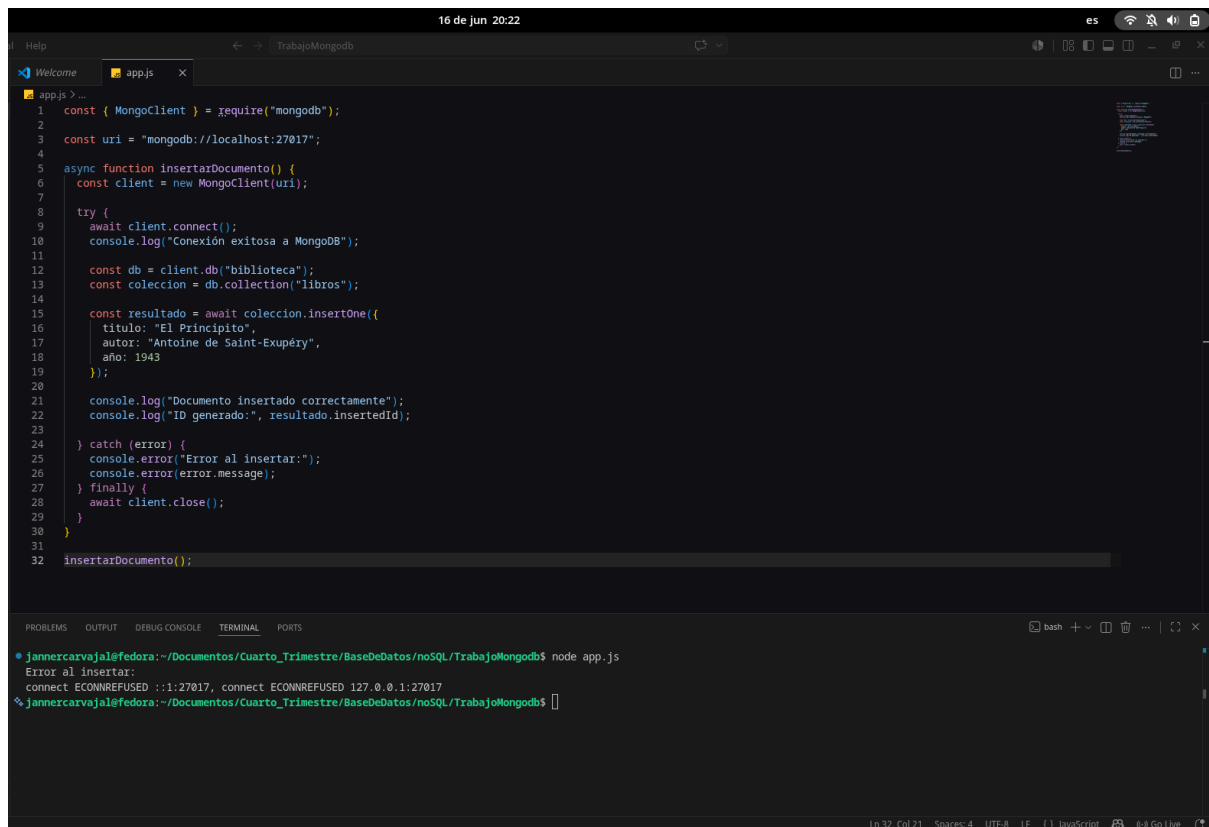
```

1  const { MongoClient } = require("mongodb");
2
3  const uri = "mongodb://localhost:27017";
4
5  async function insertarDocumento() {
6    const client = new MongoClient(uri);
7
8    try {
9      await client.connect();
10     console.log("Conexión exitosa a MongoDB");
11
12     const db = client.db("biblioteca");
13     const collection = db.collection("libros");
14
15     const resultado = await collection.insertOne({
16       titulo: "El Principito",
17       autor: "Antoine de Saint-Exupéry",
18       año: 1943
19     });
20
21     console.log("Documento insertado correctamente");
22     console.log("ID generado:", resultado.insertedId);
23   } catch (error) {
24     console.error("Error al insertar:");
25     console.error(error.message);
26   } finally {
27     await client.close();
28   }
29 }
30
31
32 insertarDocumento();
  
```

```

jannercarvajal@fedora:~/Documentos/Cuarto_Trimestre/BaseDeDatos/noSQL/TrabajoMongoDB$ node app.js
Conexión exitosa a MongoDB
Documento insertado correctamente
ID generado: new ObjectId('6a31f56f42b9c9f5e9a84ff2')
jannercarvajal@fedora:~/Documentos/Cuarto_Trimestre/BaseDeDatos/noSQL/TrabajoMongoDB$
  
```

	name	Container ID	Image	Ports	CPU (%)	Last started	Actions
	mongodb	3a80dec199a0	mongo:8	27017:27017	N/A	5 minutes ago	



```
16 de jun 20:22
es
Welcome
app.js x
app.js > ...
1 const { MongoClient } = require("mongodb");
2
3 const uri = "mongodb://localhost:27017";
4
5 async function insertarDocumento() {
6   const client = new MongoClient(uri);
7
8   try {
9     await client.connect();
10    console.log("Conexión exitosa a MongoDB");
11
12    const db = client.db("biblioteca");
13    const coleccion = db.collection("libros");
14
15    const resultado = await coleccion.insertOne({
16      titulo: "El Principito",
17      autor: "Antoine de Saint-Exupéry",
18      año: 1943
19    });
20
21    console.log("Documento insertado correctamente");
22    console.log("ID generado:", resultado.insertedId);
23
24  } catch (error) {
25    console.error("Error al insertar:");
26    console.error(error.message);
27  } finally {
28    await client.close();
29  }
30 }
31
32 insertarDocumento();
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

• jannercaavajal@fedora:~/Documentos/Cuarto_Trimestre/BaseDeDatos/noSQL/TrabajoMongoDB\$ node app.js

Error al insertar:
connect ECONNREFUSED ::1:27017, connect ECONNREFUSED 127.0.0.1:27017

% jannercaavajal@fedora:~/Documentos/Cuarto_Trimestre/BaseDeDatos/noSQL/TrabajoMongoDB\$

Ln 32, Col 21 Spaces: 4 UTF-8 LF JavaScript 6-8 Go Live

- Active el servidor y ahora en el mismo programa inserte 2 documentos, uno de ellos debe tener el id 1000 y el otro el 2000, capture el mensaje de la bases de datos.



```
mongosh mongodb://127.0.0.1:27017/

biblioteca> db.libros.insertMany([
|   {
|     _id: 1000,
|     titulo: "MongoDB Básico",
|     autor: "Janner"
|   },
|   {
|     _id: 2000,
|     titulo: "MongoDB Intermedio",
|     autor: "Janner"
|   }
| ])
{ acknowledged: true, insertedIds: { '0': 1000, '1': 2000 } }
biblioteca>
```

- ```
test> db.libros.insertMany([
| {
| _id: 4000,
| titulo: "Introducción a NoSQL",
| autor: "Janner Carvajal"
| },
| {
| _id: 5000,
| titulo: "MongoDB Avanzado",
| autor: "Janner Carvajal"
| }
|])
Uncaught:
MongoBulkWriteError: connect ECONNREFUSED 127.0.0.1:27017
Result: BulkWriteResult {
 insertedCount: 0,
 matchedCount: 0,
 modifiedCount: 0,
 deletedCount: 0,
 upsertedCount: 0,
 upsertedIds: {},
 insertedIds: { '0': 4000, '1': 5000 }
}
Write Errors: []
test>
```

- ```

Welcome
app.js
app.js > insertarDocumento
5  async function insertarDocumento() {
6    const client = new MongoClient(uri);
7
8    try {
9      await client.connect();
10     console.log("Conexión exitosa a MongoDB");
11
12     const db = client.db("biblioteca");
13     const coleccion = db.collection("libros");
14
15     const resultado = await coleccion.insertMany([
16       {
17         _id: 1000,
18         titulo: "Libro NoSQL 1",
19         autor: "Janner"
20       },
21       {
22         _id: 2000,
23         titulo: "Libro NoSQL 2",
24         autor: "Janner"
25       },
26       {
27         _id: 3000,
28         titulo: "Libro NoSQL 3",
29         autor: "Janner"
30       }
31     ]);
32
33     console.log("Documentos insertados correctamente");
34     console.log(resultado.insertedIds);
35
36   } catch (error) {
37     console.error("Error al insertar:");
38     console.error(error.message);
39   } finally {
40
41   }
42 }
43
44 export { insertarDocumento };

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

bash + ~
jannercarvajal@fedora:~/Documentos/Cuarto_Trimestre/BaseDeDatos/noSQL/TrabajoMongoDb$ node app.js
Conexión exitosa a MongoDB
Error al insertar:
E11000 duplicate key error collection: biblioteca.libros index: _id_dup key: { _id: 1000 }
jannercarvajal@fedora:~/Documentos/Cuarto_Trimestre/BaseDeDatos/noSQL/TrabajoMongoDb$

```

La operación no se completó porque MongoDB detectó que el identificador 1000 ya existía en la colección "libros". Debido a que el campo `_id` funciona como clave única para cada documento, la base de datos impide la inserción de registros con valores repetidos.